



**University of
East London**

SCHOOL OF ARCHITECTURE, COMPUTING AND ENGINEERING
Department of Computing and Digital Technologies

Classical-to-Quantum Code Transpiler Using Transformer-Based Neural Machine Translation

**Aditya Prakash
2669037**

A dissertation submitted in partial fulfilment of the degree of
BSc (Hons) in Computer Science.

Supervisor: Lucian Duta

CN6000

May 4, 2026

Acknowledgements

First and foremost, I would like to thank my mentor and supervisor, Mr Lucian Duta, for his constant support, guidance, and encouragement, which have been instrumental in shaping this research. His insightful feedback, rigorous standards, and patience throughout this journey have profoundly influenced both this work and my development as a researcher.

Secondly, I wish to extend my thanks to Dr Aloysius A. Edoh, whose trust and willingness to assist with experimental setups were invaluable. I am also indebted to him for gaining knowledge in resources critical to model implementation and data engineering.

I wish to acknowledge my fellow researchers at the University of East London for the stimulating discussions, collaborative spirit, and moral support through the inevitable challenges of research work. Our intellectual exchanges and camaraderie created a supportive community that enriched this academic pursuit.

On a personal note, I am profoundly grateful to my family for their unconditional love and encouragement. To my parents, who instilled in me the value of education and perseverance; to my colleagues, whose experience, knowledge, and professional support carried me through the most technical implementation; and to my friends, who provided necessary distractions and reminders of life beyond research.

Abstract

Purpose: The amount of development and code written using AI has significantly increased and even more developers are adopting AI in their workflow. AI is mostly used in writing documentation, testing and writing code. However, the reliability on content created by AI is low among developers due to the way how generative AI is trained and work. This is because generative models like GPTs are trained on publicly available information which is low quality and misinformation. This renders the models to create low quality outputs that might contain several security vulnerabilities and lack quality. Security is vital for production systems when it comes to high confidentiality, integrity and availability along with compliance. The project aims to analyse AI generated code for security vulnerabilities and code quality using machine learning techniques.

Design/Methods/Approach: This study initially analyses secondary dataset for vulnerable C/C++ functions labelled with CWEs associated with them to identify correlation among function patterns and CWE categories. This analysis helps identify the factors that effect or contributes to different vulnerabilities and based on function parameters like variable definition and conditions. The implementation covers the training of the fine-tune LLM and how the model evaluates in identification of CWEs in source code. The implementation has several stages starting from data preparation for training, the training, model creating and running, and inferencing. After the model is created, it is served using Ollama and the inferencing is done using a REST API built using Go and a VS Code extension is built to access this while sending the current code to the model for analysis.

Findings: The study highlight the issues in generated code and why it cannot be moved to production before sanitization and review using existing research. This paper concludes that a machine learning based static code analysis will act as a layer of security to identify the vulnerabilities in source code and raise the issue for the developer to address which makes fixing vulnerabilities faster and efficient.

Originality: Considering the research gap of machine learning based static code analysis tool, this study provided a tool that can be used in static code analysis. The use of machine learning makes this tool identify complex patterns in source code to identify vulnerabilities and label them with CWE categories. This model is distributed as a global standard `ModelFile` that can be served using different platforms like locally using Ollama, AWS Sage Maker or Azure AI Studio.

Contents

Abstract	iii
Contents	iii
List of Tables	vi
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Proposed Solution	2
1.4 Aim and Objectives	3
1.5 Document Structure	3
2 Literature Review	5
2.1 Introduction	5
2.2 Quantum Software Engineering and AI	5
2.3 Foundations of Transformers in Code Analysis and Generation	5
2.4 Quantum Circuit Representation and OpenQASM	6
2.5 PyTorch, Frameworks, and Hybrid Systems	7
2.6 Superposition, Entanglement, and Quantum Data Encoding	7
2.7 AI-Driven Code Conversion: Empirical and Theoretical Insights	8
2.8 Limitations of Current Methods and Open Challenges	8
2.9 Evaluation Criteria	9
2.10 Gaps in the Literature	9
3 Methodology	10
3.1 Introduction	10
3.2 Approach	10
3.3 Implementation	11
3.3.1 System Development Life-Cycle	11
3.3.2 Project Management Method	12
3.3.3 Tools and Technologies	13
3.4 Research Approach	14

3.4.1	Overall Research Design	14
3.4.2	Specific Analysis Method	15
3.4.3	Types of Analysis Employed	16
3.5	Challenges and Limitations	17
3.5.1	Implementation Challenges	17
3.5.2	Research Approach Limitations	18
3.6	Ethical Considerations	18
4	Results & Analysis	20
4.1	Introduction	20
4.2	Training Data Analysis	20
4.2.1	Purpose	20
4.2.2	Relevance of the Dataset	20
4.2.3	Structure of the Data	21
4.2.4	Operation Distribution	22
4.2.5	Tokenisation Analysis	22
4.2.6	Sequence Length Distribution	23
4.3	Model Training Results	23
4.3.1	Training Configuration	23
4.3.2	Training Progress	24
4.3.3	Per-Operation Accuracy	24
4.4	Implementation Outcomes	24
4.4.1	System Architecture	25
4.4.2	Hybrid Conversion Strategy	26
4.4.3	Web Application Interface	26
4.4.4	Conversion Workflow	26
4.4.5	Quantum Circuit Execution	27
4.4.6	Circuit Visualisation	27
4.4.7	QASM to Qiskit Conversion	27
4.4.8	API Performance	28
4.4.9	Model Information Panel	28
4.4.10	AI Chatbot Assistant	29
4.4.11	Project Management	29
4.5	Conclusion	29
5	Evaluation	30

5.1	Introduction	30
5.2	Research Findings	30
5.3	Results	31
5.3.1	Model Performance Analysis	31
5.3.2	Training Efficiency	32
5.3.3	Comparison of Conversion Approaches	32
5.4	Implementation	33
5.4.1	Dataset Generation	33
5.4.2	Tokenisation Strategy	33
5.4.3	Model Architecture Decisions	34
5.4.4	Post-Processing Pipeline	34
5.4.5	Issues and Challenges	34
5.5	Project Objectives	35
6	Conclusion	37
6.1	Introduction	37
6.2	Achieved Aims and Objectives	37
6.3	Critique and Limitations	38
6.4	Lessons Learned	39
6.5	Future Work	40
6.6	Greater Significance	41
	References	43

List of Tables

4.1	The features and description of the training dataset.	21
4.2	Number of unique variants and augmented training samples per quantum operation.	22
4.3	Training hyperparameters and configuration.	23
4.4	Final training, validation, and test metrics.	24
4.5	Per-operation token-level accuracy on the test set (376 samples).	25
4.6	Comparison of conversion approaches and their accuracy.	26
4.7	QASM to Qiskit gate mapping examples.	28
4.8	API endpoint response times.	28

Chapter 1: Introduction

1.1 Introduction

This chapter introduces the research by providing the motivation for undertaking the study and how the research was performed to achieve the final solution. The primary focus of this study is to design and implement a classical-to-quantum code transpiler using transformer-based neural machine translation. The growing interest in quantum computing and the difficulty classical programmers face in writing quantum circuits motivates this research.

1.2 Motivation

In recent years, there has been a significant increase in quantum computing research and development, with major technology companies investing heavily in quantum hardware and software. However, a critical barrier to the widespread adoption of quantum computing is the steep learning curve associated with quantum programming. Classical programmers must understand fundamentally different computational paradigms — including superposition, entanglement, and unitary transformations — before they can write quantum circuits.

Existing quantum programming frameworks such as Qiskit, Cirq, and Q# require developers to have deep knowledge of quantum gate-level operations. This creates a significant gap between the large number of classical programmers and the small number of quantum computing experts. Bridging this gap through automated code translation tools would accelerate the adoption of quantum computing and enable classical developers to leverage quantum advantages without requiring extensive retraining.

Neural machine translation has demonstrated remarkable success in translating between natural languages, achieving near-human performance in many language pairs. Since programming language translation shares structural properties with natural language translation — both involve mapping sequences from a source domain to a target domain while preserving semantic meaning — transformer-based models present a promising approach for automating the conversion of classical code into quantum circuits.

1.3 Proposed Solution

The solution proposed in this research is to develop a transformer-based sequence-to-sequence model that translates classical Python code into OpenQASM 2.0 quantum circuit descriptions. The model is trained on a deterministically generated dataset of verified Python-to-QASM pairs covering 13 quantum operations, including arithmetic computations (addition, subtraction, multiplication, bitwise operations) and quantum algorithms (Grover's search, Bernstein-Vazirani, Deutsch-Jozsa, teleportation, Bell states, QFT, and swap test).

The system employs a hybrid approach where deterministic circuit generators handle known operations with guaranteed correctness, while the trained transformer model serves as a fallback for novel input formulations. This is complemented by a full-stack web application with a React frontend, FastAPI backend, and Qiskit integration for circuit execution and simulation, providing an accessible interface for classical-to-quantum code transpilation.

1.4 Aim and Objectives

Aim

To design and implement a classical-to-quantum code transpiler using transformer-based neural machine translation that converts classical Python code into executable quantum circuits.

Objectives

1. To review works of literature on quantum computing, neural machine translation, and existing quantum programming tools to establish the theoretical foundation.
2. To design and implement a deterministic dataset generation pipeline producing verified Python-to-QASM training pairs across multiple quantum operations.
3. To design, train, and evaluate a transformer encoder-decoder model for classical-to-quantum code translation.
4. To implement a hybrid conversion system combining deterministic generators with neural model inference for optimal accuracy.
5. To develop a full-stack web application providing an interactive interface for code conversion, circuit visualisation, and quantum simulation.
6. To evaluate the system's performance through per-operation accuracy analysis and functional testing of the complete pipeline.
7. To document the findings, reflect on limitations, and suggest directions for future research and development.

1.5 Document Structure

The research is structured into six chapters. Chapter two provides the background and literature review, covering quantum computing fundamentals, neural machine translation, and existing quantum programming tools. Chapter three discusses the methodology, including the development approach, system architecture, tools and technologies, and research design. Chapter four presents the results of the training data analysis, model performance metrics, and implementation outcomes. Chapter five evaluates the findings by discussing the research results, implementation challenges, and achievement of project objectives. Finally, chapter six

concludes the study with a summary of achievements, critique and limitations, lessons learned, and suggestions for future work.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the existing body of literature on converting classical (conventional) code to quantum representations, focusing on approaches that leverage transformer models and related AI techniques. It critically evaluates foundational theories, practical developments (such as OpenQASM, PyTorch-based code translation frameworks, and hybrid quantum-classical systems), and identifies current knowledge gaps and unresolved challenges. The aim is to position this study within the rapidly evolving field of quantum coding, bridging the disciplines of machine learning, compiler theory, and quantum computing

2.2 Quantum Software Engineering and AI

Quantum software engineering is an interdisciplinary domain that adapts classical software engineering principles to quantum computational models. Classical software is mapped into quantum circuits or assembly to harness quantum mechanics properties such as superposition and entanglement for enhanced computation (Piattini et al., 2020; Zhao). Unlike classical programming, which relies on deterministic boolean logic, quantum programming requires the orchestration of probabilistic unitary transformations. AI-driven methods, especially transformers, have shown promise in automating and optimizing this translation, supporting tasks like code parsing, semantic understanding, and the generation of quantum-native instructions (Wakeham, 2024). Fingerhuth et al. (2018) emphasizes that the scarcity of quantum algorithmic experts necessitates such automated tools to democratize access to quantum resources. Furthermore, Ali et al. (2022) argues that the integration of machine learning into the quantum software lifecycle is essential for handling the complexity of circuit synthesis and verification.

2.3 Foundations of Transformers in Code Analysis and Generation

Transformers are now the backbone for understanding complex patterns in text, source code, and instructions. Their multi-head self-attention mechanisms support powerful non-local de-

pendency modeling—crucial for tasks that involve structural and semantic mapping, such as code translation or synthesizing new representations (Le et al., 2022). In the context of source code, these models capture the hierarchical and syntactical relationships inherent in programming languages, enabling high-fidelity translation between languages (Chen et al., 2021).

In the quantum context, both classical and quantum transformer models have been studied for their ability to parse, translate, and generate quantum logic. Specialized designs, such as Parameterized Quantum Circuit (PQC) and Quantum Linear Algebra (QLA) based transformers, have been proposed for NISQ-era hardware. demonstrated that Large Language Models (LLMs) could generate syntactically correct OpenQASM code from natural language prompts, although functional correctness remains a challenge for complex algorithms. Moreover, Guo et al. (2025) introduced benchmarking frameworks specifically designed to evaluate the capability of these transformer models in generating quantum circuits, highlighting the need for domain-specific fine-tuning.

2.4 Quantum Circuit Representation and OpenQASM

Core to quantum code execution is the quantum circuit abstraction: an ordered set of qubit instructions, measurements, and classical controls. OpenQASM has emerged as the de facto intermediate representation (IR) language, supporting low-level gate descriptions, parameterization, subroutines, and classical-quantum control flow for execution on real and simulated quantum hardware (Caro et al., 2021; Cross et al.). When converting code, the transformer typically produces, directly or indirectly, OpenQASM or a similar intermediate representation such as the Quantum Intermediate Representation (QIR) (Javadi-Abhari et al., 2024).

Effective use of OpenQASM is integral to pipelines that process, optimize, and deploy code on frameworks like IBM Qiskit and Google Cirq (Aleksandrowicz et al., 2019). The evolution of OpenQASM 3.0 introduced dynamic circuit capabilities and pulse-level control, increasing the complexity of the translation task but also the expressivity of the target code (Cross et al.). Automated conversion systems must therefore account for these varied abstraction levels, ensuring that high-level classical logic is correctly transpiled into the appropriate quantum gate sequences (Mete et al., 2024).

2.5 PyTorch, Frameworks, and Hybrid Systems

PyTorch is central to many contemporary deep learning and hybrid quantum-classical system workflows. Libraries such as TorchQuantum and PennyLane enable modelers to design, optimize, and simulate quantum circuits using the familiar PyTorch interface while bridging to quantum hardware or simulators for deployment (Bergholm et al., 2018). These frameworks facilitate joint training, gradient-based optimization, and seamless classical-quantum model fusion, aiding transformer-based code conversion (Biamonte et al., 2017).

The integration of quantum simulation within deep learning pipelines allows for the end-to-end training of "quantum-aware" neural networks. For instance, it discusses the utility of hybrid frameworks in handling high-dimensional data processing. In the context of code conversion, these frameworks provide the necessary differentiable backend to optimize the parameters of generated quantum circuits, ensuring they approximate the desired classical function logic Weder et al. (2025).

2.6 Superposition, Entanglement, and Quantum Data Encoding

Quantum states, often referred to as superstates in the context of high-dimensional representation, underpin the expressive power of quantum circuits. Superposition allows for the representation of an exponential number of configurations, while entanglement enables non-classical dependencies that are crucial for quantum algorithms (Nielsen and Chuang, 2010). Translating classical inputs and logic into quantum circuits necessarily involves encoding methods, such as amplitude, angle, and basis encoding, with each encoding affecting circuit size, depth, trainability, and execution on quantum hardware (LaRose and Coyle, 2020).

Lloyd et al. (2020) highlights that the "input problem"—efficiently loading classical data into quantum states—remains a significant bottleneck. For code conversion, this implies that a transformer must not only translate the algorithm but also determine the optimal data encoding strategy. The analyzed generalization performance of these encodings suggests that the choice of embedding significantly impacts the learnability of the resulting quantum model. Consequently, modern conversion tools must balance the trade-off between circuit depth and the expressivity of the encoded state (Kyriienko et al., 2021).

2.7 AI-Driven Code Conversion: Empirical and Theoretical Insights

Recent research demonstrates how transformers and LLMs, trained or fine-tuned on quantum syntax (QASM/Cirq), are capable of not just translating classical algorithms but also composing new quantum circuits and subroutines. Neural approaches to code translation, including immediate code-to-quantum mapping, have been validated by Esposito et al. (2024), who introduced frameworks for translating Python directly to QASM via abstract syntax tree traversal and pattern matching.

Furthermore, the use of agentic AI systems paired with quantum toolchains for end-to-end pipeline automation is gaining traction. Kerenidis and Cherrat (2025) demonstrated agents capable of algorithmic discovery, suggesting that AI can move beyond translation to optimization. Benchmarks of quality, correctness, and resource efficiency relative to hand-coded or compiler-generated baseline circuits are essential for validating these approaches (Guo et al., 2025). Finally, the integration of quantum error correction and optimization logic generated from AI-informed pipelines represents the frontier of this research, with studies by Niu et al. (2019) and Fösel et al. (2021) utilizing reinforcement learning to optimize quantum error correction codes and circuit structures.

2.8 Limitations of Current Methods and Open Challenges

While transformer-based and hybrid AI-code-conversion approaches have yielded promising results, significant challenges persist. Scalability and hardware constraints remain a primary hurdle; limited qubit counts, shallow circuit depths, and high error rates constrain the real-world utility of generated code. Data encoding and embedding bottlenecks also present difficulties, as efficient and expressive encoding from classical to quantum domains remains an area of active research with no universal solution Preskill (2018).

Optimization and trainability issues, specifically barren plateaus and vanishing gradients, pose major obstacles to performance scaling in parameterized quantum circuits (Cer, 2017). Wang et al. (2021) notes that these optimization landscapes become exponentially difficult to navigate as system size increases. Finally, correctness and verification remain non-trivial; ensuring the functional equivalence and formal correctness of AI-generated quantum code is difficult due to the probabilistic nature of quantum mechanics. While hybrid formal/ML verification techniques are emerging, they are not yet mature enough for industrial application (McClean et al.,

2018).

2.9 Evaluation Criteria

Evaluation typically combines traditional ML metrics with specific quantum circuit metrics. Code and circuit correctness are paramount, assessing functionality, equivalence, and determinism (Guo et al., 2025). Quantum resource usage is evaluated through qubit count, circuit depth, and gate count, which serve as proxies for the feasibility of execution on NISQ devices (Caro et al., 2021). Execution efficiency on simulators and hardware is also critical, often measured by total runtime and fidelity. Finally, standard benchmarks such as F1 score, accuracy, quantum volume, and error rates provide quantitative measures for comparison against state-of-the-art baselines (Cross et al.).

2.10 Gaps in the Literature

Synthesis of the reviewed sources reveals several critical gaps in the current body of knowledge. Firstly, there is a distinct lack of large, real-world validated datasets for classical-to-quantum code conversion, forcing researchers to rely on synthetic or limited benchmark sets (Guo et al., 2025). Secondly, there is a pressing need for standardized, interpretable benchmarks that can simultaneously assess functional correctness and quantum-specific performance metrics like gate fidelity and coherence time usage. Thirdly, there is limited exploration of generalization across varied algorithmic classes and codebases; most current studies focus on narrow domains such as optimization or specific linear algebra subroutines (Esposito et al., 2024). Finally, there is insufficient study regarding the integration of error-correction integration and hardware-aware code synthesis within the transformer generation process, leaving a gap between theoretically generated code and physically executable circuits (Fösel et al., 2021).

Chapter 3: Methodology

3.1 Introduction

This chapter presents the complete methodology adopted for the design, implementation, and evaluation of a conventional-to-quantum code transpiler based on transformer models. It explains the overall approach guiding the work, the development life-cycle and project management methods used to construct the system, the tools and technologies employed, and the research design underpinning the empirical analysis. The chapter is organised to provide a coherent narrative that begins with high-level methodological choices, proceeds through detailed implementation strategies and experimental procedures, and concludes with a rigorous discussion of challenges, limitations, and ethical issues. The overarching objective is to offer a transparent and fully justified account of how the system was built, how it was evaluated, and why the chosen methods are appropriate for the research aims.

3.2 Approach

The project follows a dual methodology that integrates a software engineering perspective with a structured empirical research perspective. From the software engineering side, the work is driven by iterative development and continuous feedback between requirements, design, implementation, and testing. This reflects the inherently exploratory nature of transformer-based models and quantum circuit generation, where key design decisions such as model architecture, tokenisation schemes, and quantum representations cannot be fixed a priori and must be refined through experimentation. From the empirical research side, the project adopts a quantitative, experiment-driven paradigm in which system variants are treated as experimental conditions tested against structured datasets under controlled settings. This paradigm supports the collection of numerical evidence on translation quality, circuit characteristics, and model behaviour, enabling systematic comparison and replication.

The overarching methodology is motivated by the need to manage uncertainty and complexity in both the deep learning and quantum computing components. Transformer models require extensive experimentation with hyperparameters, architecture depth, attention mechanisms, and training schedules, while quantum circuit generation demands careful design of interme-

diate representations and execution back-ends. An iterative, feedback-oriented development cycle allows these factors to be adjusted as new evidence is obtained, rather than committing to a rigid plan that may prove suboptimal. At the same time, the structured research orientation introduces discipline in experiment design, data partitioning, metric selection, and result interpretation, thereby ensuring that conclusions drawn from the experiments rest on a clear and reproducible methodological foundation.

3.3 Implementation

3.3.1 System Development Life-Cycle

The system is developed using an Agile-inspired, incremental life-cycle model. Instead of treating requirements, design, implementation, and evaluation as strictly sequential phases, the work is organised into repeated iterations, each producing a working increment of the transpiler or its supporting infrastructure. This choice is shaped by the fact that many crucial decisions cannot be validated solely through theoretical reasoning, but must be tested empirically. For example, the suitability of a particular tokenisation scheme for quantum assembly code or the effectiveness of a given transformer depth in capturing control-flow patterns can only be assessed after concrete models are trained and evaluated. An incremental life-cycle makes it possible to start with a minimal viable pipeline and then refine and extend it as new insights emerge.

Each iteration begins with a refinement of requirements and scope. At this stage, the subset of the conventional language to be handled by the transpiler is clarified, as is the corresponding quantum abstraction to be targeted. The scope may, for instance, initially focus on simple arithmetic and basic control structures before expanding to include more complex patterns such as nested loops or function calls. Following requirement refinement, attention shifts to architectural and design modelling. Here, the pipeline is decomposed into components for code parsing, data preprocessing, model encoding and decoding, and quantum circuit construction. The relationships between these components are specified in order to ensure that data flows correctly from input program to generated circuit and that each component has a clearly defined responsibility. This design work incorporates lessons learned from previous iterations, such as the need for additional preprocessing steps or stricter post-processing checks.

Once the design of a given iteration is defined, data engineering and preparation are undertaken. This involves collecting or generating new examples of code pairs where necessary,

cleaning and normalising existing examples, and transforming them into the sequence formats required by the transformer architecture. Particular care is taken to ensure that the test set remains unseen during model training, so that evaluation results reflect genuine generalisation rather than memorisation. After the data pipeline is stable, implementation proceeds. This stage includes coding the model architectures, writing training scripts, integrating parsing and tokenisation components, and implementing the quantum circuit generation logic that converts model outputs into executable representations. When implementation is complete for an iteration, training and tuning are conducted, with models trained on the current dataset and hyperparameters adjusted based on validation performance.

The iteration concludes with evaluation and analysis. The system is subjected to a battery of tests designed to measure syntactic correctness of generated quantum code, functional correctness of resulting circuits, and quantitative circuit properties such as depth and number of qubits. The results are analysed to identify strengths and weaknesses, such as constructs that the model handles reliably and scenarios where it fails. These findings inform the planning of the next iteration, where requirements, design, data, and models are adjusted accordingly. A life-cycle diagram illustrating this process is included as a figure in the chapter to make the iterative structure explicit and to emphasise the feedback loops from evaluation back to design and data.

3.3.2 Project Management Method

Project management throughout the work follows a Kanban-style approach tailored to an individual research and development context. Rather than fixed-length sprints, the method revolves around a continuously updated board that visualises the status of tasks and constrains the amount of work in progress. Tasks represent concrete units of effort, such as designing the grammar for the supported subset of the conventional language, implementing a new transformer configuration, constructing a new experimental dataset, or running a specific group of evaluation experiments. Each task progresses through conceptual stages such as being planned, currently under development, undergoing review or validation, and finally completed.

This visual and flow-based method provides several advantages for a research-oriented project. First, it accommodates the variability and uncertainty of experimental tasks. Some experiments may complete quickly or reveal that a particular approach is ineffective, requiring rapid re-prioritisation of subsequent work. Others may encounter unforeseen obstacles, such as training instabilities or integration issues with quantum simulators, that necessitate additional tasks for

debugging and refinement. The Kanban approach allows such changes to be reflected immediately in the board, avoiding the rigidity that fixed sprint plans can impose. Second, by limiting the number of tasks that can be in progress at any given time, the method encourages focus and reduces context switching, which is particularly important when experiments demand careful configuration and interpretation.

In parallel with the Kanban board, a high-level timeline is maintained in the form of a Gantt-style plan. This timeline identifies major milestones such as the completion of the initial system architecture, the availability of the first fully functional prototype, the completion of baseline experiments, the execution of extended experimental campaigns, and the final synthesis of results for the dissertation. The timeline is not treated as immutable but is used to ensure that the project remains aligned with the overall academic schedule. When experimental results suggest new avenues for investigation or when some tasks prove more time-consuming than anticipated, the timeline is updated accordingly to reallocate effort while preserving critical submission deadlines.

3.3.3 Tools and Technologies

The implementation of the transpiler relies on a combination of programming languages, deep learning frameworks, quantum software frameworks, and supporting development tools. A high-level general-purpose programming language is used as the primary implementation language due to its strong ecosystem for machine learning, data processing, and scientific computing. Within this language, a modern deep learning framework provides the infrastructure for implementing transformer-based architectures, including facilities for constructing encoder–decoder models, batch processing of sequence data, automatic differentiation, and GPU-accelerated training. This framework supports rapid prototyping of model variants and simplifies the implementation of complex attention mechanisms and training regimes.

On the quantum side, a dedicated quantum software framework is employed to construct, manipulate, and execute quantum circuits. The framework offers an intermediate representation language or assembly language for quantum operations, enabling the generated circuits to be expressed in a platform-neutral form. It also includes circuit-building APIs that can be used to translate the textual quantum code output by the transformer into structured circuit objects suitable for simulation. Through this framework, the project gains access to simulators capable of executing circuits and returning measurement statistics, which are essential for assessing functional correctness. In addition, the framework may provide tools for basic circuit optimisation and for mapping circuits to hypothetical hardware back-ends, although the main emphasis

in this work remains on simulator-based evaluation.

Supporting tools play an important role in enabling reproducible, efficient development. Data processing libraries are used to ingest code corpora, perform tokenisation for both source and target languages, and construct datasets for training and evaluation. Plotting and visualisation libraries assist in analysing training curves, metric distributions, and exploratory statistics on the dataset and model outputs. Integrated development environments and code editors provide debugging and linting capabilities, while version control systems track all code changes and experimental configurations. Configuration files and logging mechanisms are used to record the details of each experiment, including model parameters, dataset splits, random seeds, and evaluation results, thus ensuring that experiments can be replicated or revisited later.

The architecture is designed with scalability and extensibility in mind. The pipeline is conceptually partitioned into three layers: a frontend that parses and preprocesses conventional code into a representation suitable for model input, a core model that performs the sequence-to-sequence translation, and a backend that converts model outputs into quantum circuits and executes them. This separation of concerns allows each layer to be modified or replaced independently. For instance, the frontend can be extended to support additional language constructs, the model can be upgraded to a different transformer variant or a hybrid classical–quantum architecture, and the backend can be reconfigured to target different quantum frameworks or hardware devices as they become available.

3.4 Research Approach

3.4.1 Overall Research Design

The research component of the project adopts a predominantly quantitative design. The core objective is to evaluate how effectively the transformer-based transpiler can translate conventional programs into quantum circuits, and to understand the trade-offs involved in terms of accuracy, resource usage, and robustness. To this end, the study is structured around a series of experiments that use numerical metrics computed over a systematically constructed dataset. The design emphasises reproducibility: datasets are partitioned into training, validation, and test subsets; models are trained under controlled conditions; and all evaluation metrics are computed on held-out data that the model has not encountered during training.

The study primarily uses secondary data. Conventional programs and their quantum coun-

terparts are derived from a combination of existing examples and systematically constructed cases. Some examples are inspired by canonical quantum algorithms and typical programming patterns, while others are synthetically generated to exercise specific structures such as nested control flow or parameterised operations. The reliance on secondary data means that the research does not involve human participants or the collection of personal information. Instead, it focuses on operationalising the translation task in a way that allows objective measurement. The experimental design also includes the use of baselines, where feasible, so that the performance of the transformer-based approach can be compared to simpler methods or to hand-crafted circuits.

3.4.2 Specific Analysis Method

The analysis method is grounded in supervised learning and quantitative evaluation. The dataset is composed of pairs of input programs in the conventional language and corresponding target representations in the quantum assembly or intermediate representation. Each pair is assumed to represent the same underlying computation, enabling the assessment of both syntactic and semantic fidelity. The dataset is carefully constructed to include a range of difficulty levels. At the simplest level, it incorporates small programs that map to basic circuits, such as single-qubit rotations or simple entangling operations. More complex examples incorporate multiple qubits, control structures, and compound operations. The inclusion of multiple complexity levels supports the investigation of how model performance scales with program complexity.

Models are trained using supervised sequence-to-sequence learning. The conventional program, after tokenisation and possibly additional preprocessing steps such as abstract syntax tree linearisation, serves as the input sequence to the transformer encoder, while the quantum representation serves as the target output sequence for the decoder. Training is performed using mini-batch optimisation, with loss functions defined over the token sequences. Hyperparameters such as learning rate, batch size, number of training epochs, and model depth are chosen based on preliminary experiments and validation performance. During training, validation data is used to monitor overfitting and to guide early stopping or learning rate schedules.

Evaluation is conducted using several complementary metrics. Syntactic correctness is assessed by attempting to parse the generated quantum representations and checking whether they conform to the expected grammar of the intermediate or assembly language. Semantic or functional correctness is evaluated by constructing circuits from the generated code and ex-

ecuting them on a simulator, then comparing the resulting measurement statistics or output states to those of reference circuits derived from the target representations. Resource usage is quantified by metrics such as the number of qubits in each circuit, the depth of the circuit (measured as the length of the longest gate sequence), and the total number of gates. In addition, sequence-level metrics such as exact match proportion and token-level accuracy are computed to quantify how often the generated text matches the target exactly or partially. These metrics together provide a multifaceted view of performance that captures both textual and operational aspects of the translation.

3.4.3 Types of Analysis Employed

Several types of analysis are used to interpret the results of the experiments. Exploratory analysis is applied to the dataset and model outputs before formal conclusions are drawn. For the dataset, this includes examining the distributions of program lengths, the frequency of different syntactic constructs, and the balance between simpler and more complex examples. For the model outputs, exploratory analysis involves inspecting typical success and failure cases, identifying patterns such as recurring mistakes in translating particular constructs, and visualising the distribution of evaluation metrics across the test set. These findings guide refinements in dataset composition, model design, and training procedures.

Correlational analysis is then used to investigate relationships between variables of interest. For example, the analysis may examine how translation accuracy varies with program length, the depth of control-flow nesting, or the number of qubits in the target circuit. It may also explore how different model sizes or training regimes relate to metric outcomes, such as whether deeper models consistently produce circuits with lower error rates or reduced resource consumption. By quantifying these relationships, correlational analysis helps to identify factors that are particularly influential for performance and to understand trade-offs, such as increases in accuracy at the cost of higher computational demand.

Comparative analysis is conducted to evaluate the relative merits of alternative approaches and configurations. Within the transformer-based family, different architectures, tokenisation strategies, or training settings can be compared in terms of their impact on syntactic and semantic correctness as well as resource usage. Where feasible, baseline methods are also included in the comparison. These may involve rule-based or template-based translators that map specific patterns of conventional code to predetermined quantum circuits. Comparing the transformer-based approach against these baselines clarifies whether the complexity and computational cost of training large models yields tangible benefits in terms of translation quality

and circuit properties.

3.5 Challenges and Limitations

3.5.1 Implementation Challenges

The implementation of a conventional-to-quantum code transpiler that integrates deep learning with quantum software frameworks presents several technical challenges. One major challenge arises from the need to connect components that operate at different abstraction levels. The transformer model processes token sequences and produces textual quantum code, while the quantum software framework expects structured circuit objects. Bridging this gap requires robust parsing and post-processing mechanisms that can convert raw model outputs into syntactically valid, semantically meaningful circuits. Errors or ambiguities in this conversion step can compromise both syntactic correctness and functional behaviour.

Another challenge concerns performance and resource constraints. Training transformer models demands significant computational resources, particularly when model depth and width are increased or when large datasets are used. When multiple model variants and hyperparameter configurations are explored, training time can become a critical limiting factor. Similarly, simulating quantum circuits is resource-intensive, especially for circuits involving many qubits or deep gate sequences. Memory limitations and execution time constraints can restrict the size and number of circuits that can be feasibly evaluated, which in turn may limit the scope of the experiments. These constraints necessitate careful design choices regarding model size, dataset composition, and the complexity of example programs included in the evaluation.

A further set of challenges arises from the constrained feature coverage of the transpiler. Because not all aspects of a general-purpose conventional language map naturally to quantum circuits, the system must focus on a carefully defined subset of the language. This requires clear decisions about which constructs to support and how to handle constructs that fall outside the supported subset. In practice, this may mean ignoring certain features, approximating them, or flagging them as unsupported. Each choice carries implications for usability and for the interpretation of results. In addition, the quantum side may impose its own constraints. For instance, some abstract operations may not have straightforward circuit implementations within the chosen gate set or may require circuit depths that are impractical for simulation.

3.5.2 Research Approach Limitations

The research design is limited by the nature and availability of data. The dataset used for training and evaluation, while carefully constructed, is necessarily finite and may not fully represent the diversity of real-world codebases. It may be biased toward certain types of algorithms or coding styles, particularly if many examples are derived from textbook or reference implementations. As a result, the measured performance may not generalise to arbitrary conventional programs outside the scope of the dataset. The reliance on secondary and synthetic data also means that some examples might be simplified or idealised relative to real-world usage, further affecting generalisability.

Evaluation methodology introduces additional limitations. Functional correctness is assessed using simulator-based comparisons between generated circuits and reference circuits, but these tests inevitably cover a finite set of inputs or measurement scenarios. Complex programs with large input spaces cannot be exhaustively tested, and some errors may remain undetected if they only manifest under rarely encountered conditions. Moreover, simulators approximate the behaviour of ideal or noise-modelled quantum devices, but they do not fully capture all nuances of actual hardware, such as specific error channels, calibration issues, or temporal drift. Consequently, circuits that perform well in simulation may behave differently on physical devices, a limitation that must be acknowledged when interpreting results.

Finally, the findings are specific to the particular models, datasets, and back-ends employed. Different transformer architectures, alternative tokenisation schemes, larger or more diverse datasets, or different quantum frameworks could lead to different performance profiles. The conclusions drawn about the feasibility and effectiveness of transformer-based conventional-to-quantum translation should therefore be understood as contingent on the methodological choices made in this project, rather than as definitive statements about all possible instantiations of such systems.

3.6 Ethical Considerations

Ethical considerations in this project relate primarily to data use, computational resource consumption, and the broader implications of automated quantum code generation. With respect to data, the project focuses on source code and circuit artefacts rather than on human subjects or personal information. Nevertheless, care is taken to ensure that any external code used in constructing the dataset complies with appropriate licensing conditions and does not contain sensitive or proprietary content. When examples originate from publicly available sources,

proper attribution is respected in line with academic and legal norms. Generated data, such as synthetic code examples, is created specifically for research purposes and does not derive from confidential sources.

The substantial computational demands of training transformer models and simulating quantum circuits raise questions about responsible use of resources. To address this, experiments are planned to maximise the knowledge gained per unit of computation. Redundant or marginally informative experiments are avoided, and trained models are reused where possible to minimise repeated training. When possible, hyperparameter searches are constrained to carefully chosen ranges rather than exhaustive sweeps. These practices reflect an awareness that computation bears both financial and environmental costs and that these costs should be justified by the research value of the work.

Finally, the development of tools that automate quantum code generation has broader ethical and practical implications. On one hand, such tools can lower barriers to entry by enabling users without deep expertise in quantum programming to experiment with quantum algorithms. On the other hand, they may produce outputs that users cannot easily interpret or verify, potentially leading to misplaced trust in generated circuits. To mitigate these risks, the methodology adopted in this project emphasises transparency about system limitations and encourages users to treat the transpiler as an aid rather than a definitive authority. The chapter therefore emphasises the need for critical evaluation of generated circuits and for clear documentation of the circumstances in which the system performs reliably and the scenarios where it is prone to failure.

Chapter 4: Results & Analysis

4.1 Introduction

This chapter discusses the results of the experiments conducted using the classical-to-quantum code transpiler and analyses the data generated during training and evaluation. The analysis is structured around the training dataset characteristics, model training performance, per-operation accuracy metrics, and the functional outcomes of the complete system. Furthermore, the implementation outcomes including the web-based interface, API backend, and quantum circuit execution are presented and evaluated.

4.2 Training Data Analysis

4.2.1 Purpose

The purpose of this analysis is to understand the dataset used for training the transformer model and to evaluate the quality and diversity of the generated training examples. Since the dataset is deterministically generated from verified quantum algorithm implementations, the analysis confirms the correctness, balance, and coverage of quantum operations represented in the training corpus.

4.2.2 Relevance of the Dataset

4.2.2.1 Original Purpose

The dataset was created specifically for training a sequence-to-sequence transformer model to translate classical Python code into OpenQASM 2.0 quantum circuit descriptions. Unlike scraped or manually curated datasets, all training examples are algorithmically generated from mathematically verified quantum circuit implementations, ensuring correctness of every input-output pair.

4.2.2.2 How the Data was Generated

The data was generated using deterministic circuit generators implemented in Python. Each generator corresponds to a specific quantum operation (e.g., Cuccaro addition, Grover’s search, Bernstein-Vazirani) and produces valid Python-QASM pairs for all possible input combinations within the defined bit-width. The generation process ensures that every circuit is functionally correct by construction, as the generators implement well-established quantum algorithms.

4.2.2.3 Credibility

The credibility of the dataset is confirmed by two factors. First, each training example is generated from verified quantum algorithm implementations that produce mathematically correct circuits. Second, every entry in the dataset is marked with a `verified: true` flag, indicating that the generated QASM circuit was validated against expected quantum behaviour. The deterministic nature of generation eliminates human error in data labelling.

4.2.3 Structure of the Data

The dataset contains 3,584 training examples distributed across 13 quantum operations. Each example is stored in JSON format with the features described in Table 4.1.

Feature	Type	Description
python	text	The classical Python code representing the operation.
qasm	text	The corresponding OpenQASM 2.0 quantum circuit.
operation	text	A description of the specific operation (e.g., “7 + 5”).
algorithm_type	factor	The quantum algorithm category (e.g., <code>cuccaro_addition</code>).
expected_result	number	The expected computational result of the operation.
verified	boolean	Whether the circuit has been verified for correctness.

Table 4.1: The features and description of the training dataset.

The dataset is split into training, validation, and test sets following an 80/10/10 ratio, resulting in 2,857 training samples, 351 validation samples, and 376 test samples.

4.2.4 Operation Distribution

Operations with naturally fewer unique variants (e.g., Bell states with 4 variants, teleportation with 3 variants) are augmented through repetition to achieve approximately 256 samples per operation, ensuring balanced representation during training. The augmentation strategy ensures that the model does not develop bias towards operations with more unique combinations (e.g., addition with 256 unique pairs vs. teleportation with 3 states). Table 4.2 provides the exact counts per operation.

Operation	Unique Variants	Augmented Count
Addition (Cuccaro)	256	256
Subtraction	256	512
Multiplication	16	256
Bitwise AND	256	256
Bitwise OR	256	256
Bitwise XOR	256	256
Grover's Search	12	256
Bernstein-Vazirani	28	256
Deutsch-Jozsa	6	256
Teleportation	3	256
Bell States	4	256
QFT	28	256
Swap Test	4	256

Table 4.2: Number of unique variants and augmented training samples per quantum operation.

4.2.5 Tokenisation Analysis

The tokenisation strategy employs word-level regex tokenisation using the pattern `\w+| [^\w\s]` to split both Python and QASM code into tokens. All text is lowercased before tokenisation. The resulting vocabulary contains 220 unique tokens, including four special tokens: `<PAD>`, `<UNK>`, `<START>`, and `<END>`.

The maximum input sequence length is set to 128 tokens for Python code, while the maximum output sequence length is 768 tokens for QASM circuits. This asymmetry reflects the fact that

quantum circuits are typically more verbose than their classical equivalents, particularly for operations like the Cuccaro adder which requires extensive gate sequences.

4.2.6 Sequence Length Distribution

Analysis of the training data reveals significant variation in sequence lengths across operations. Arithmetic operations such as addition and subtraction produce the longest QASM sequences (approximately 400-600 tokens) due to the Cuccaro ripple-carry architecture requiring multiple MAJ and UMA gate sequences. In contrast, simpler operations like Bell state preparation produce sequences of approximately 20-30 tokens. This variation necessitates careful padding strategies and the use of attention masks during training.

4.3 Model Training Results

4.3.1 Training Configuration

The transformer model was trained using the configuration specified in Table 4.3. Training was conducted on a CPU-based system, with the AdamW optimiser and a combined warmup-cosine annealing learning rate schedule.

Parameter	Value
Optimiser	AdamW
Learning Rate	3×10^{-4}
Weight Decay	0.01
Batch Size	16
Loss Function	CrossEntropyLoss (label smoothing = 0.1)
Gradient Clipping	max_norm = 1.0
LR Schedule	5 epochs warmup + cosine annealing
Early Stopping	Patience = 20 epochs
Training Epochs	8

Table 4.3: Training hyperparameters and configuration.

4.3.2 Training Progress

The model converged rapidly, reaching 99.59% validation accuracy within 8 epochs before early stopping was triggered. The training and validation loss decreased consistently over the training period, demonstrating convergence without overfitting. The final training metrics are summarised in Table 4.4.

Metric	Value
Final Training Loss	0.8909
Final Training Accuracy	99.16%
Final Validation Loss	0.8843
Final Validation Accuracy	99.59%
Test Set Accuracy (Token-level)	99.57%

Table 4.4: Final training, validation, and test metrics.

The fact that validation accuracy (99.59%) marginally exceeds training accuracy (99.16%) suggests that the model generalises well and that label smoothing effectively prevents overfitting to the training distribution.

4.3.3 Per-Operation Accuracy

Table 4.5 presents the token-level accuracy achieved on the test set for each of the 13 quantum operations. The results demonstrate consistently high accuracy across all operations, with teleportation achieving perfect accuracy and Grover’s search showing the lowest (but still high) accuracy at 98.85%.

The lower accuracy for Grover’s search can be attributed to its greater circuit complexity, involving multi-controlled-Z gates and diffusion operators that produce longer and more intricate QASM sequences. Despite this, the 98.85% token-level accuracy remains remarkably high for neural sequence generation.

4.4 Implementation Outcomes

The final implementation is a full-stack web application comprising a trained PyTorch transformer model (11.17 million parameters), a FastAPI backend serving as the conversion and execution engine, and a React-based frontend providing an interactive three-panel interface for

Operation	Accuracy (%)	Notes
Teleportation	100.00	Bell pair + corrections
Swap Test	99.84	Fredkin-based state comparison
Multiplication	99.77	Toffoli network (2-bit)
Deutsch-Jozsa	99.75	Constant vs balanced oracle
Bell States	99.72	2-qubit entangled pairs
Bitwise AND	99.65	Toffoli per bit
Addition	99.62	Cuccaro ripple-carry
Bitwise OR	99.61	CX + CCX per bit
Bitwise XOR	99.59	CX per bit
QFT	99.50	Controlled-phase gates
Subtraction	99.37	Two's complement + Cuccaro
Bernstein-Vazirani	99.26	Hidden string discovery
Grover's Search	98.85	Oracle + diffusion
Overall	99.57	

Table 4.5: Per-operation token-level accuracy on the test set (376 samples).

code conversion, visualisation, and execution. The system architecture follows a client-server pattern where the frontend communicates with the backend via RESTful API calls.

4.4.1 System Architecture

The system is composed of three primary components that work together to provide end-to-end classical-to-quantum code transpilation. The architecture follows a layered design:

1. **Frontend Layer:** React 19 with TypeScript, providing the user interface with code editors, circuit visualisation, and result display.
2. **API Layer:** FastAPI (Python) handling HTTP requests, model inference orchestration, and quantum circuit execution.
3. **Model Layer:** PyTorch transformer performing the actual Python-to-QASM translation.
4. **Execution Layer:** Qiskit with AerSimulator for circuit simulation and measurement.

4.4.2 Hybrid Conversion Strategy

A key design decision in the implementation is the hybrid approach to code conversion. The system first attempts to use deterministic circuit generators for recognised operations before falling back to the trained neural model. This ensures 100% accuracy for known operations while leveraging the model's generalisation capability for novel inputs.

Approach	Applicable Operations	Accuracy
Deterministic Generators	All 13 known operations	100%
Trained Transformer Model	All operations (fallback)	99.57%
Hybrid (Generators + Model)	All operations	99.5–100%

Table 4.6: Comparison of conversion approaches and their accuracy.

4.4.3 Web Application Interface

The web application provides a three-panel editor layout that simultaneously displays the classical Python input, the generated OpenQASM 2.0 circuit, and the auto-converted Qiskit Python code. The interface includes the following features:

- Syntax highlighting for Python, QASM, and Qiskit code
- Real-time line numbering in all editors
- Copy and download functionality for each panel
- Responsive layout adapting to different screen sizes
- Operation type detection and display

4.4.4 Conversion Workflow

The conversion process begins when the user enters classical Python code and initiates the conversion. The system performs the following steps:

1. The frontend sends the Python code to the backend via `POST /convert`.
2. The backend detects the operation type using keyword-based analysis.

3. If a deterministic generator exists for the detected operation, it generates the QASM directly.
4. Otherwise, the trained model tokenises the input, performs greedy autoregressive decoding, and reconstructs the QASM output.
5. The generated QASM is validated and post-processed (register expansion, gate deduplication, syntax correction).
6. The frontend receives the QASM and automatically converts it to Qiskit Python code using the client-side converter.

4.4.5 Quantum Circuit Execution

The system provides the capability to execute generated quantum circuits on a simulated quantum computer. When the user clicks the execute button, the QASM circuit is sent to the backend where it is parsed by Qiskit, transpiled, and run on the AerSimulator with a configurable number of shots (default: 1024). The execution results include:

- Measurement counts for each basis state
- Probability distribution across states
- Circuit statistics (depth, gate count, qubit count)
- Binary and decimal result visualisation for arithmetic operations

4.4.6 Circuit Visualisation

The `QuantumCircuitVisualizer` component renders a visual representation of the quantum circuit from the generated QASM code. This visualisation displays the qubit wires, gate operations, and measurement operations in a standard circuit diagram format, allowing users to visually verify the generated circuit structure.

4.4.7 QASM to Qiskit Conversion

The frontend includes a comprehensive QASM-to-Qiskit converter implemented in TypeScript (437 lines). This utility parses the generated QASM circuit and produces executable Qiskit Python code, supporting all standard gates including parameterised rotations, controlled operations, and multi-qubit gates. Table 4.7 shows a subset of the supported gate mappings.

QASM Gate	Qiskit Method	Description
<code>h q[0]</code>	<code>qc.h(0)</code>	Hadamard gate
<code>cx q[0], q[1]</code>	<code>qc.cx(0, 1)</code>	CNOT gate
<code>ccx q[0], q[1], q[2]</code>	<code>qc.ccx(0, 1, 2)</code>	Toffoli gate
<code>cp(pi/4) q[0], q[1]</code>	<code>qc.cp(pi/4, 0, 1)</code>	Controlled-phase
<code>rx(pi/2) q[0]</code>	<code>qc.rx(pi/2, 0)</code>	X-rotation
<code>swap q[0], q[1]</code>	<code>qc.swap(0, 1)</code>	SWAP gate
<code>cswap q[0], q[1], q[2]</code>	<code>qc.cswap(0, 1, 2)</code>	Fredkin gate

Table 4.7: QASM to Qiskit gate mapping examples.

4.4.8 API Performance

The FastAPI backend provides consistent response times for both conversion and execution requests. Table 4.8 summarises the typical response times observed during testing.

Endpoint	Response Time	Notes
<code>POST /convert (generator)</code>	<100ms	Deterministic path
<code>POST /convert (model)</code>	2–10s	Neural inference
<code>POST /execute</code>	<500ms	Qiskit simulation
<code>GET /model-info</code>	<50ms	Cached metadata
<code>GET /health</code>	<10ms	Status check

Table 4.8: API endpoint response times.

The significant difference between deterministic generator response times (<100ms) and neural model inference (2–10 seconds) highlights the computational cost of autoregressive transformer decoding on CPU hardware. However, both paths are within acceptable limits for an interactive application.

4.4.9 Model Information Panel

The application includes a dedicated model information panel that displays the architecture specifications, training metrics, per-operation accuracy, and provides access to test data examples. This transparency allows users to understand the model’s capabilities and limitations.

4.4.10 AI Chatbot Assistant

The system integrates a quantum computing expert chatbot powered by the DeepSeek API. This assistant provides context-aware help, answering questions about quantum computing concepts, explaining generated circuits, and suggesting improvements. The chatbot uses server-sent events for streaming responses, providing a responsive conversational experience.

4.4.11 Project Management

The project was managed using Git and GitHub for version control. The commit history throughout the implementation period reflects the iterative development of the model training pipeline, API backend, and frontend application.

4.5 Conclusion

The classical-to-quantum code transpiler achieves a 99.57% token-level accuracy on the test set across 13 quantum operations. The hybrid approach combining deterministic generators with a trained transformer model ensures both correctness and flexibility. The full-stack implementation provides an accessible interface for converting classical Python code into executable quantum circuits, with real-time visualisation, Qiskit code generation, and quantum simulation capabilities. The system demonstrates that transformer-based neural machine translation is a viable approach for bridging classical and quantum programming paradigms.

Chapter 5: Evaluation

5.1 Introduction

This chapter evaluates the key findings of the research by analysing the effectiveness of the transformer-based approach for classical-to-quantum code transpilation. The evaluation is structured into four sections: the first discusses the research findings in the context of existing literature, the second evaluates the model’s results and their implications, the third analyses the implementation outcomes and challenges encountered, and the fourth revisits the project objectives to assess the degree to which they have been achieved.

5.2 Research Findings

As established in the literature review, the barrier between classical and quantum programming remains a significant challenge for the wider adoption of quantum computing. The research confirms that classical programmers face substantial difficulty in translating algorithmic intent into quantum circuit representations, primarily due to the fundamentally different computational paradigms involved. Quantum circuits operate on qubits through unitary transformations rather than the sequential instruction execution familiar to classical programmers, making manual translation error-prone and time-consuming.

The literature demonstrated that neural machine translation (NMT) techniques, particularly transformer architectures, have achieved remarkable success in natural language translation tasks. This study extends that paradigm to programming language translation, specifically from classical Python to OpenQASM 2.0. The key insight from the research is that code translation shares structural properties with natural language translation — both involve mapping sequences from a source domain to a target domain while preserving semantic meaning — making transformer models a theoretically sound choice for this task.

One of the objectives of this study was to evaluate the feasibility of using transformer-based models for quantum circuit generation. The findings confirm that a standard encoder-decoder transformer with 6 layers, 8 attention heads, and 256-dimensional embeddings is sufficient to learn the mapping between classical arithmetic operations and their quantum circuit equiv-

alents with near-perfect accuracy. This is significant because it demonstrates that relatively modest model sizes (11.17 million parameters) can capture the structural relationships between classical code and quantum circuits, without requiring the billions of parameters typical of large language models.

The research also revealed that deterministic dataset generation from verified quantum algorithm implementations provides a superior training signal compared to web-scraped or manually curated datasets. The deterministic approach guarantees correctness of every training example, eliminates labelling errors, and enables systematic coverage of the input space. This finding has broader implications for training domain-specific translation models where correctness is paramount.

Furthermore, the study identified that a hybrid approach combining deterministic generators with neural models provides the optimal balance between accuracy and generalisation. Deterministic generators ensure 100% correctness for known operations, while the neural model provides the flexibility to handle variations in input formatting and potentially generalise to unseen operation types.

5.3 Results

The results of this study demonstrate that a transformer-based neural machine translation model can achieve 99.57% token-level accuracy in translating classical Python code to OpenQASM 2.0 quantum circuits. This accuracy was achieved across 13 distinct quantum operations spanning arithmetic computations, quantum algorithms, and quantum information protocols.

5.3.1 Model Performance Analysis

The model achieved its highest accuracy on quantum teleportation (100%) and its lowest on Grover's search (98.85%). This performance differential can be attributed to the varying complexity of the target circuits. Teleportation circuits follow a fixed, short template (3 qubits, approximately 10 gates), making them highly predictable for the model. In contrast, Grover's search involves variable-length oracle constructions and diffusion operators with multi-controlled gates, producing longer and more structurally diverse circuits that challenge the model's sequence generation capabilities.

An important distinction must be made between token-level accuracy and functional circuit accuracy. While the model achieves 99.57% token-level accuracy, this does not directly trans-

late to 99.57% of circuits being functionally correct. A single incorrect token in a critical position (e.g., a wrong qubit index or gate type) can render an entire circuit semantically invalid. However, the post-processing validation pipeline mitigates this by detecting and correcting common errors such as out-of-range qubit indices and syntax malformations.

5.3.2 Training Efficiency

A notable finding is the rapid convergence of the model, reaching near-perfect accuracy within only 8 training epochs. This can be attributed to several factors:

- The deterministic dataset provides clean, consistent training signals without noise or labelling errors.
- The relatively small vocabulary (220 tokens) reduces the complexity of the output space.
- The structural regularity of quantum circuits (headers, register declarations, gate sequences, measurements) provides strong positional patterns for the model to learn.
- Label smoothing (0.1) and the cosine annealing learning rate schedule prevent premature convergence to local optima.

The training efficiency demonstrates that domain-specific translation tasks with well-structured data can be solved with modest computational resources, without requiring extensive hyperparameter tuning or massive datasets.

5.3.3 Comparison of Conversion Approaches

The evaluation of the hybrid conversion strategy reveals a clear trade-off between deterministic and neural approaches. Deterministic generators provide guaranteed correctness but are limited to pre-programmed operations. The neural model offers flexibility but introduces a small probability of error. The hybrid system leverages the strengths of both: deterministic generators handle known operations with perfect accuracy, while the model serves as a fallback for edge cases or novel formulations.

The response time analysis further differentiates the approaches. Deterministic generators respond in under 100 milliseconds, while neural inference requires 2–10 seconds due to autoregressive token generation on CPU hardware. For an interactive application, this suggests that

deterministic generators should be preferred whenever possible, with the neural model reserved for cases where no generator matches the input.

5.4 Implementation

5.4.1 Dataset Generation

The dataset generation pipeline produces training examples by exhaustively enumerating all valid input combinations for each quantum operation within defined bit-widths. For 4-bit arithmetic, this yields 256 unique addition pairs (16×16), 256 subtraction pairs, and 16 multiplication pairs (4×4 for 2-bit inputs). Quantum algorithms with fewer natural variants are augmented through repetition to ensure balanced training.

The deterministic generation approach eliminates several common challenges in machine learning dataset preparation:

- No data cleaning is required since all examples are generated from verified implementations.
- No labelling errors exist since the QASM output is produced by the same algorithm that defines the correct circuit.
- Class balance is explicitly controlled through the augmentation strategy.
- The data is inherently reproducible, enabling exact replication of experiments.

5.4.2 Tokenisation Strategy

The word-level regex tokenisation strategy was chosen over character-level or subword tokenisation (e.g., BPE) for several reasons. First, quantum assembly code has a well-defined and limited vocabulary where meaningful units align with word boundaries (gate names, qubit indices, keywords). Second, the small vocabulary size (220 tokens) makes word-level tokenisation computationally efficient. Third, word-level tokens preserve semantic meaning, making it easier for the model to learn gate-level patterns rather than character-level reconstructions.

However, this tokenisation choice introduces challenges in QASM reconstruction, as tokens like `q`, `[`, `0`, and `]` must be reassembled into `q[0]`. The post-processing pipeline handles this reconstruction with specific rules for brackets, decimals, comments, and operators.

5.4.3 Model Architecture Decisions

The choice of a standard PyTorch `nn.Transformer` with 6 encoder and 6 decoder layers represents a balance between model capacity and training efficiency. The 256-dimensional embedding space and 1024-dimensional feed-forward layers provide sufficient representation power for the 220-token vocabulary without excessive parameterisation.

Sinusoidal positional encoding was chosen over learned positional embeddings because the fixed mathematical formulation generalises to sequence positions not seen during training, which is important given the variable-length nature of quantum circuits. The maximum position length of 1024 accommodates even the longest circuits in the training set (Cuccaro addition circuits reaching approximately 600 tokens).

5.4.4 Post-Processing Pipeline

The post-processing pipeline is critical to the system's practical utility. Raw model output often contains minor formatting issues that do not affect token-level accuracy metrics but would prevent circuit execution. The pipeline performs:

- **Register expansion:** Automatically increases `qreg` and `creg` sizes if generated gates reference qubit indices beyond the declared range.
- **Conditional conversion:** Converts bit-level conditionals to register-level format required by OpenQASM 2.0.
- **Gate deduplication:** Removes duplicate gate operations that occasionally appear in model output.
- **Syntax reconstruction:** Reassembles tokenised elements into valid QASM syntax (brackets, decimals, operators).
- **Length capping:** Limits output to 5000 characters to prevent degenerate cases of repetitive generation.

5.4.5 Issues and Challenges

Several issues and challenges were encountered during the research and implementation of the transpiler.

The most significant challenge was managing the variable-length nature of quantum circuits. While simple operations like Bell state preparation require only a handful of gates, complex operations like Cuccaro addition produce circuits with over 25 gates across 9 qubits. This variability makes it difficult to set appropriate maximum sequence lengths — too short truncates complex circuits, while too long wastes computational resources on padding.

A second challenge involved the greedy decoding strategy. While greedy decoding (selecting the highest-probability token at each step) is computationally efficient, it does not guarantee globally optimal sequences. Beam search would theoretically improve output quality but at significant computational cost for long sequences. The decision to use greedy decoding was made based on the empirical observation that it produces sufficiently accurate results (99.57%) while maintaining reasonable inference times.

The frontend implementation presented challenges related to the real-time conversion of QASM to Qiskit Python code. The client-side converter must handle all possible gate types, parametrised rotations, and measurement patterns while producing syntactically valid Python. Edge cases involving conditional measurements and multi-parameter gates required extensive testing and iterative refinement.

Another challenge was the inference latency on CPU hardware. The autoregressive nature of transformer decoding means that each output token requires a full forward pass through the decoder, resulting in 2–10 second response times for complex circuits. This is acceptable for an educational tool but would require GPU acceleration or model distillation for production deployment.

5.5 Project Objectives

This study set out to develop a transformer-based neural machine translation system for converting classical Python code into quantum circuits. Returning to the aim of the study — to design and implement a classical-to-quantum code transpiler using transformer-based neural machine translation — the findings demonstrate that this approach is not only feasible but highly effective, achieving near-perfect accuracy on a diverse set of quantum operations.

The research objectives can be evaluated as follows:

1. **Literature review on quantum computing and NMT:** The literature review established

the theoretical foundation for applying transformer models to code translation, identifying the structural parallels between natural language and programming language translation.

2. **Evaluation of existing quantum programming tools:** The review of existing tools (Qiskit, Cirq, Q#) demonstrated that while these frameworks provide powerful circuit construction capabilities, they require significant quantum computing expertise, motivating the need for automated transpilation.
3. **Dataset design and generation:** A deterministic dataset generation pipeline was successfully implemented, producing 3,584 verified training examples across 13 quantum operations with guaranteed correctness.
4. **Model architecture design and training:** A transformer encoder-decoder model with 11.17 million parameters was designed and trained, achieving 99.57% token-level accuracy on the test set within 8 epochs.
5. **System implementation:** A complete full-stack application was developed comprising a FastAPI backend, React frontend, and Qiskit integration for circuit execution and simulation.
6. **Evaluation and testing:** Comprehensive evaluation was conducted across all 13 operations, with per-operation accuracy analysis, response time measurements, and functional testing of the complete conversion pipeline.
7. **Documentation and reflection:** The implementation, results, and limitations have been documented with suggestions for future improvements and research directions.

All project objectives have been achieved. The transpiler successfully converts classical Python code into executable quantum circuits with high accuracy, and the web-based interface provides an accessible tool for both education and practical quantum circuit generation.

Chapter 6: Conclusion

6.1 Introduction

This chapter summarises the key findings of the study, evaluates whether the original aim and objectives have been achieved, discusses the critiques and limitations of the research and implementation, presents lessons learned during the project, suggests directions for future work, and concludes with a reflection on the greater significance of the research.

6.2 Achieved Aims and Objectives

The aim of the present research was to design and implement a classical-to-quantum code transpiler using transformer-based neural machine translation. The research has demonstrated that a standard encoder-decoder transformer architecture can learn the mapping between classical Python code and OpenQASM 2.0 quantum circuits with 99.57% token-level accuracy across 13 distinct quantum operations. This result confirms that neural machine translation is a viable and effective approach for bridging classical and quantum programming paradigms.

The most significant finding to emerge from this study is that relatively modest transformer models (11.17 million parameters, 6 encoder/decoder layers) are sufficient to capture the structural relationships between classical arithmetic operations and their quantum circuit equivalents. This challenges the assumption that code generation tasks necessarily require large language models with billions of parameters, demonstrating instead that focused, domain-specific models trained on high-quality data can achieve near-perfect accuracy with minimal computational resources.

The second major finding is that deterministic dataset generation from verified quantum algorithm implementations provides a superior foundation for training translation models compared to scraped or manually curated data. The 3,584 verified training examples, generated from mathematically correct quantum circuit implementations, enabled rapid model convergence (8 epochs) without the noise, labelling errors, or class imbalance issues that typically plague machine learning datasets.

This study also successfully developed a complete full-stack application that operationalises the trained model. The system comprises a FastAPI backend serving the transformer model and Qiskit-based circuit execution, a React frontend providing an interactive three-panel interface, and a comprehensive QASM-to-Qiskit converter. The application demonstrates that the research findings can be translated into a practical, usable tool for quantum circuit generation.

Returning to the research objectives, all seven objectives have been achieved:

1. The literature review established the theoretical foundation for transformer-based code translation and surveyed existing quantum programming tools.
2. A deterministic dataset generation pipeline was implemented covering 13 quantum operations.
3. A transformer model was designed, trained, and evaluated, achieving 99.57% accuracy.
4. A hybrid conversion system combining deterministic generators with neural inference was implemented.
5. A full-stack web application was developed for interactive code transpilation.
6. Comprehensive evaluation was conducted with per-operation accuracy analysis.
7. The system was documented with reflection on limitations and future directions.

6.3 Critique and Limitations

Despite the high accuracy achieved, several limitations of the current implementation must be acknowledged.

The most significant limitation is the scope of supported operations. The model is trained on 13 specific quantum operations with fixed bit-widths (4-bit arithmetic, 2-bit multiplication). It cannot generalise to arbitrary classical programs, recursive algorithms, or operations outside its training distribution. The transpiler is, in its current form, a specialised translation tool for a defined set of quantum algorithms rather than a general-purpose compiler.

A second limitation relates to the distinction between token-level accuracy and functional correctness. While 99.57% of individual tokens are generated correctly, a single incorrect token can render an entire circuit semantically invalid. The actual rate of completely correct circuits

is likely lower than the token-level metric suggests, though the post-processing pipeline mitigates many common errors.

The current system operates exclusively on CPU hardware, resulting in inference times of 2–10 seconds for neural model conversion. While acceptable for educational use, this latency would be prohibitive for integration into automated development workflows or continuous integration pipelines where sub-second response times are expected.

The tokenisation strategy, while effective for the current vocabulary, treats source code as a flat sequence of tokens without preserving the hierarchical structure of programs. This limits the model's ability to understand nested control flow, function composition, or variable scoping that would be necessary for handling more complex classical programs.

Finally, the maximum output sequence length of 768 tokens constrains the complexity of circuits that can be generated. Operations requiring more gates than can fit within this limit (e.g., higher bit-width arithmetic or multi-iteration algorithms) would be truncated, producing incomplete circuits.

6.4 Lessons Learned

Several important lessons emerged from the research and implementation process.

First, the quality of training data matters more than quantity for domain-specific translation tasks. The 3,584 deterministically generated, verified examples produced better results than would be expected from a much larger but noisy dataset. This reinforces the principle that data quality should be prioritised over data volume, particularly in specialised domains where correctness is verifiable.

Second, the hybrid approach of combining deterministic generators with neural models proved to be a pragmatic engineering decision. Pure neural approaches sacrifice guaranteed correctness, while pure deterministic approaches sacrifice flexibility. The hybrid system captures the best of both worlds and reflects a practical design philosophy for deploying machine learning in safety-critical applications.

Third, post-processing is essential for neural code generation systems. Raw model output, while statistically accurate at the token level, often contains minor formatting or structural issues that prevent execution. Investing in robust post-processing pipelines significantly im-

proves the practical utility of the system without requiring improvements to the underlying model.

Fourth, the choice of intermediate representation significantly impacts system design. OpenQASM 2.0 provides a well-defined, text-based representation that aligns well with sequence-to-sequence translation. However, its verbosity (especially for complex circuits) introduces challenges for token length limits that might be better addressed by more compact representations in future work.

Fifth, the iterative development approach was essential for managing the complexity of a system spanning machine learning, quantum computing, and web development. Early prototypes with minimal functionality enabled rapid feedback on architectural decisions, preventing costly rework later in the development process.

6.5 Future Work

Several directions for future research and development are suggested by the findings and limitations of this study.

The most impactful extension would be expanding the supported operation set to include additional quantum algorithms such as Shor’s algorithm, variational quantum eigensolver (VQE), quantum approximate optimisation algorithm (QAOA), and quantum error correction codes. This would require developing additional circuit generators and training the model on more diverse circuit structures, potentially necessitating a larger model capacity.

A second direction involves replacing the flat sequence tokenisation with tree-structured or graph-based input representations. Parsing classical Python code into an Abstract Syntax Tree (AST) and encoding this tree structure into the transformer input could preserve program semantics more faithfully than the current word-level tokenisation. This would enable the model to handle more complex classical programs with nested control flow and function composition.

Third, implementing beam search or other advanced decoding strategies (e.g., nucleus sampling with verification) could improve functional circuit correctness at the cost of increased inference time. A promising middle ground would be to generate multiple candidate circuits via beam search and select the one that passes Qiskit validation, effectively using the quantum simulator as a verification oracle.

Fourth, GPU-accelerated inference and model quantisation could reduce response times to sub-second levels, making the system suitable for integration into development environments and IDE plugins. Techniques such as INT8 quantisation and ONNX runtime optimisation could provide significant speedups without substantial accuracy degradation.

Fifth, extending the system to support multiple target quantum hardware backends (IBM Quantum, Google Cirq, Amazon Braket) would increase its practical utility. This could be achieved by training separate decoder heads for each target representation or by implementing a common intermediate representation with backend-specific code generators.

Finally, investigating the use of Graph Attention Networks (GAT) operating on the program's control flow graph or data dependency graph could provide a more principled approach to understanding classical code structure. Combined with the current transformer decoder for QASM generation, this could create a more powerful hybrid architecture capable of handling arbitrary classical programs.

6.6 Greater Significance

Taking the full scope of this research into consideration, the project represents a contribution towards making quantum computing more accessible to classical programmers. As quantum hardware continues to advance and quantum advantage is demonstrated for increasingly practical problems, the demand for tools that lower the barrier to quantum programming will grow.

The transformer-based transpilation approach demonstrated in this study shows that machine learning can serve as an effective bridge between classical and quantum programming paradigms. By allowing developers to express their intent in familiar Python syntax and automatically generating the corresponding quantum circuits, the system removes the need for deep expertise in quantum gate-level programming for standard operations.

Furthermore, the project contributes to the growing body of evidence that domain-specific, focused machine learning models can outperform general-purpose systems for specialised tasks. The success of a 11.17 million parameter model in achieving 99.57% accuracy challenges the trend towards ever-larger models and suggests that targeted training on high-quality data remains a powerful approach for well-defined translation tasks.

The educational value of the system should also be emphasised. By providing simultaneous views of classical code, quantum assembly, and Qiskit Python alongside circuit visualisation

and execution results, the application serves as an interactive learning tool that helps users develop intuition for the relationship between classical algorithms and their quantum implementations. This educational dimension may prove to be the most immediate practical impact of the work, contributing to the development of a quantum-literate workforce.

References

- Quantum machine learning. *Nature*, 549(7671):195–202, 2017. doi: 10.1038/nature23474.
- Gadi Aleksandrowicz et al. Qiskit: An open-source framework for quantum computing. *Zenodo*, 2019. doi: 10.5281/zenodo.2562110.
- Shaukat Ali, Tao Yue, and Rui Abreu. When software engineering meets quantum computing, 4 2022. URL <https://cacm.acm.org/research/when-software-engineering-meets-quantum-computing/>.
- Ville Bergholm et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018. doi: 10.1038/nature23474.
- Jacob Biamonte et al. Quantum machine learning. *Nature*, 549(7671):195–202, 2017. doi: 10.1038/nature23474.
- Matthias C. Caro et al. Encoding-dependent generalization bounds for parametrized quantum circuits. volume 5, page 582, 2021. doi: 10.22331/q-2021-11-17-582. URL <https://quantum-journal.org/papers/q-2021-11-17-582/>.
- Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Andrew Cross et al. Openqasm 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3Zenodo. doi: 10.5281/zenodo.5114813.
- M. Esposito et al. Classical q: Towards a translation framework to bridge classical and quantum computing. *arXiv preprint arXiv:2406.09305*, 2024.
- Mark Fingerhuth et al. Open source software in quantum computing. *PLoS ONE*, (12):e0208561, 2018. doi: 10.1371/journal.pone.0208561.
- Thomas Fösel et al. Quantum circuit optimization with deep reinforcement learning. *Physical Review X*, 11(1):011022, 2021. doi: 10.1103/PhysRevX.11.011022.
- Xiaoyu Guo, Minggu Wang, and Jianjun Zhao. Quanbench: Benchmarking quantum code generation with large language models. In *Proceedings of the IEEE/ACM Automated Software Engineering (ASE)*, 2025. URL <https://arxiv.org/abs/2510.16779>. Preprint available.

- Ali Javadi-Abhari et al. volume 6, 2024. doi: 10.1007/s42484-024-00139-1. URL <https://link.springer.com/journal/42484>.
- Iordanis Kerenidis and El-Amine Cherrat. Quantum agents for algorithmic discovery. *arXiv preprint arXiv:2510.08159*, 2025.
- Oleksandr Kyriienko et al. Solving nonlinear differential equations with differentiable quantum circuits. *Physical Review A*, 103(5):052416, 2021. doi: 10.1103/PhysRevA.103.052416.
- Ryan LaRose and Brian Coyle. Robust data encodings for quantum classifiers. *Physical Review A*, 102(3):032420, 2020. doi: 10.1103/PhysRevA.102.032420.
- Hung Le et al. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv:2207.01780*, 2022.
- Seth Lloyd et al. Quantum embeddings for machine learning. *arXiv preprint arXiv:2001.03622*, 2020.
- Jarrod R. McClean et al. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 2018. doi: 10.1038/s41467-018-07090-4.
- A. Mete et al. Model-driven quantum code generation. *Quantum Science and Technology*, 9, 2024.
- Michael A. Nielsen and Isaac L. Chuang. *Quantum computation and quantum information*. 2010. URL <https://asignaturas.df.uba.ar/tdmc-pazroncagliaschmiegelowwisniacki/wp-content/uploads/sites/50/2025/03/Quantum-Computation-and-Quantum-Information-MICHAEL-NIELSEN-ISAAC-CHUANG.pdf>.
- Murphy Yuezhen Niu et al. Universal quantum control through deep reinforcement learning. *npj Quantum Information*, 5(1):1–8, 2019. doi: 10.1038/s41534-019-0141-3.
- Mario Piattini et al. The talavera manifesto for quantum software engineering and programming. *Quantum Software Engineering*, pages 1–5, 2020.
- John Preskill. Quantum computing in the nisc era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.
- D. Wakeham. Transformers in quantum computing. *Quantum Machine Intelligence*, 6, 2024. doi: 10.1007/s42484-024-00139-1.
- S. Wang et al. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12(1):1–11, 2021. doi: 10.1038/s41467-021-27045-6.

- Benjamin Weder et al. Hardware-level interfaces for hybrid quantum-classical computing systems. *arXiv preprint arXiv:2009.07129*, 2025.
- Jianjun Zhao. Quantum software engineering: Landscapes and challenges. *arXiv preprint arXiv:2007.07*.